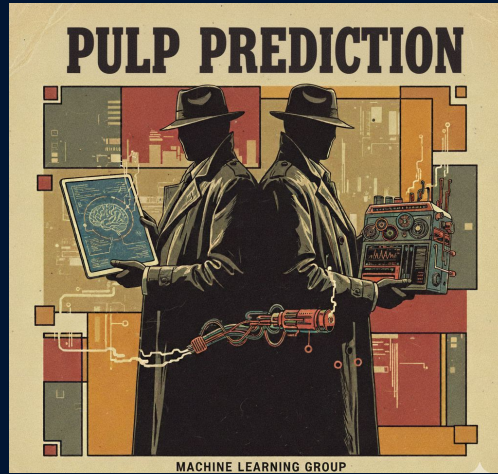




Milestone 1-4 Reflection

Team 6: Pulp Predictions

Monish, Adam, Sarah, Aravinda, Meghna



01 | Milestone

Key Design Decisions and Technical Choices

Problem: Only 35% of interactions had explicit ratings

Our Solution: Hybrid Rating Strategy

We designed a percentile-based implicit rating system using watch-time behavior:

- Top 10% watch time → ★★★★★
- Top 25% → ★★★★★
- Top 50% → ★★★★★
- Lower percentiles mapped to 1-2 stars
- Ensures fairness by basing thresholds on entire dataset distribution

Why This Matters

- Converts all 2.5M interactions into usable ratings
- Preserves behavioral meaning: longer watch -> stronger preference

Key Design Decisions and Technical Choices

- Did data stitching and feature engineering of the data received from the kafka logs
- Chose Two Models
 - ALS (Alternating Least Squares) - Implicit Feedback Model
 - Trained on watch-time matrix
 - Great for large-scale streaming platforms
 - SVD (Singular Value Decomposition) - Explicit Rating Model
 - Trained on final_rating (explicit + hybrid implicit ratings)
 - Learns dense latent factors capturing deeper taste patterns
- Enabled fallback architecture in our deployed service
- Used Docker in the M1 itself to make service persistent easy and tracking easie

Challenges Overcome

- **Continuous data collection**
 - Streamed data from Kafka and stitched logs into clean interactions
 - Ensured we always had fresh and complete data for training
- **Integrating the model into a service**
 - Turned offline models into a working API with fast responses
 - Built fallback logic to keep the service running even if one model fails
- **Handling noisy and sparse rating data**
 - Only 35% of users gave ratings -> needed a smarter strategy
 - Built percentile-based rating system to fill missing ratings



02 | Milestone

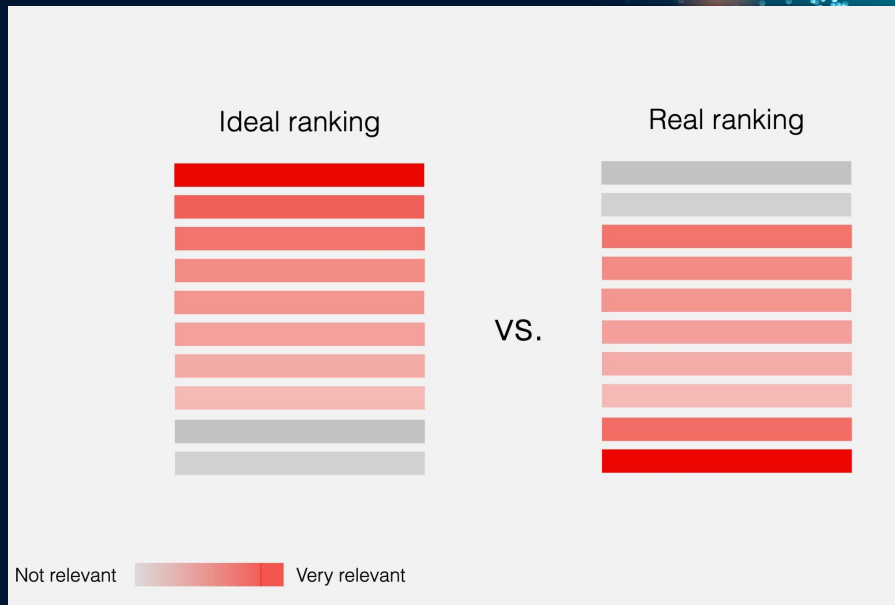
Key Design Decisions and Technical Choices

Offline Evaluation

- We wanted to add a **metric** that would account things **not captured** by recall and precision
 - they only care if a movie is in the top 20, not where it's ranked (Set Based)

We wanted a metric that reward the model for putting more relevant items higher in the recommendation list.

- We added **nDCG@20** (Normalized Discounted Cumulative Gain for top 20 results)



Key Design Decisions and Technical Choices



General Practices

Build a model -> Update the service -> Push image of the service the **docker repository**

- To maintain application level provenance so if anything goes wrong we can revert and keep the service running

Used GitHub Actions to reduce load on the VM

- Ran the workflow on Githubs end
 - build a Docker image
 - run tests
 - run linting
 - package artifacts
 - do CI checks

The VM was able to focus on **hosting** the model service and **inference**.

Challenges Overcome



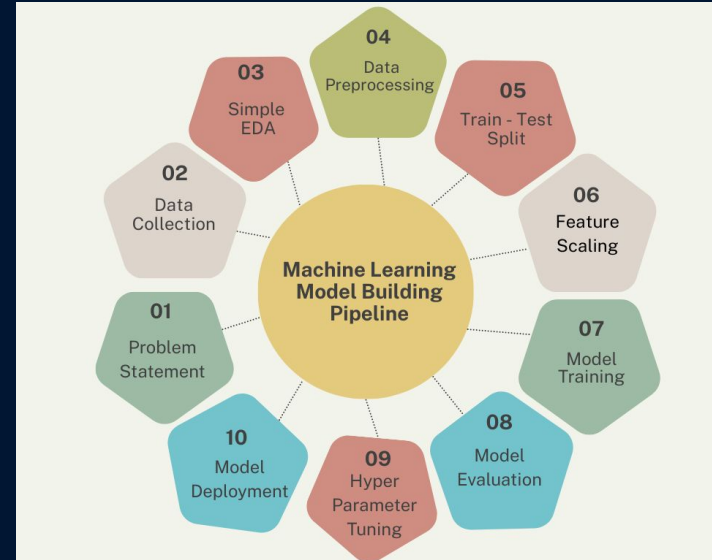
Refactoring

We assumed going from a monolithic to a modular structure would be generally straightforward

We were mistaken

We struggled to maintain functionality as we implemented different parts of the pipeline.

- Global variables
- File paths and working directory
- Error handling



Reflection and Lessons Learnt

1. The pipeline we created was created in an ad-hoc nature.

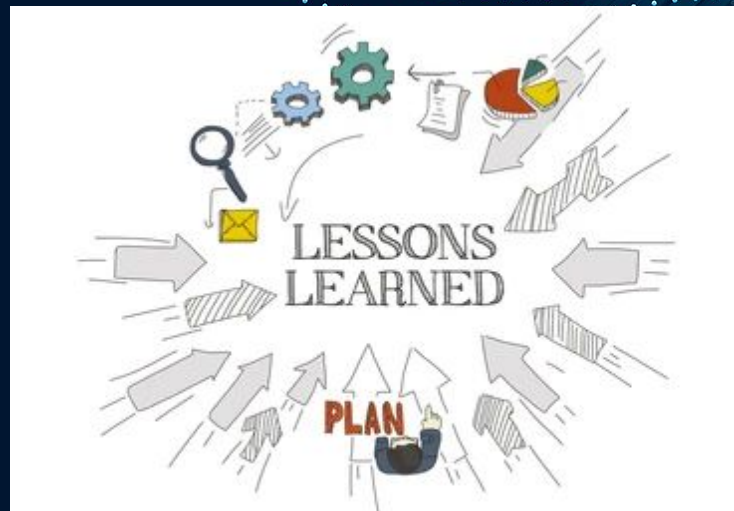
- Which become **technical debt** in later milestones

2. The pipeline was too complex

- Sometimes less is more.
 - There were a lot of components that were adding any value or were serving as bloat. Made it harder to debug and update

3. Bad branching, merging and PR strategies

- Led us to create 1 single PR that contained EVERYTHING



Commits 35 Checks 10 Files changed 56

+7,954 -204,827 ■■■■■



03 | **Milestone**

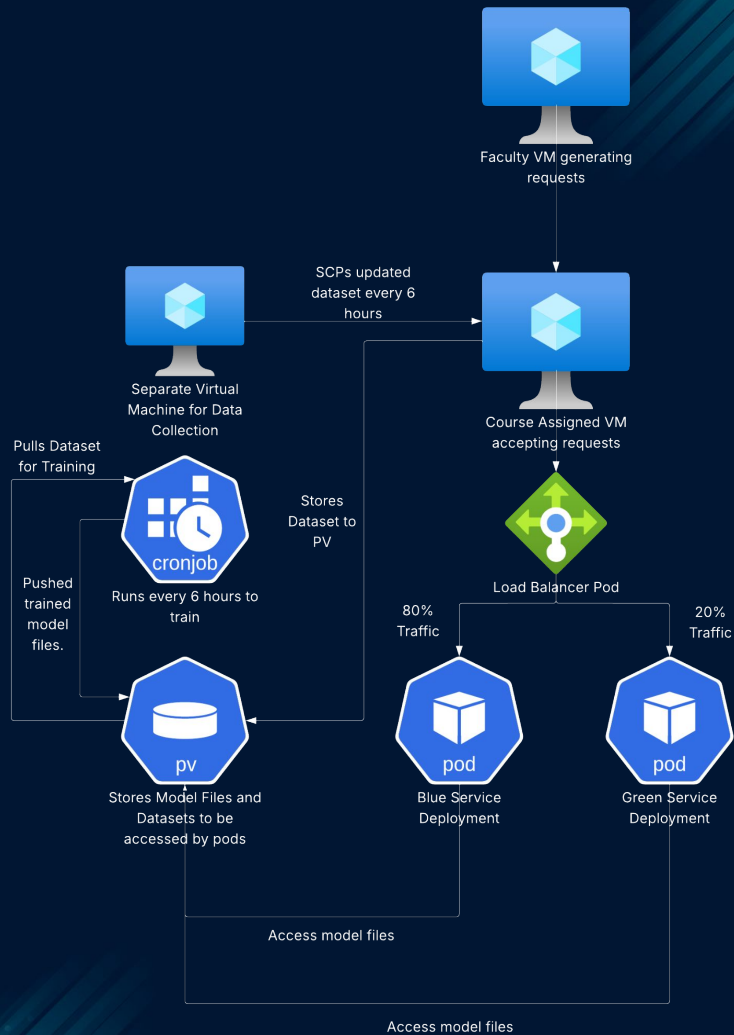
Key Design Decisions and Technical Choices

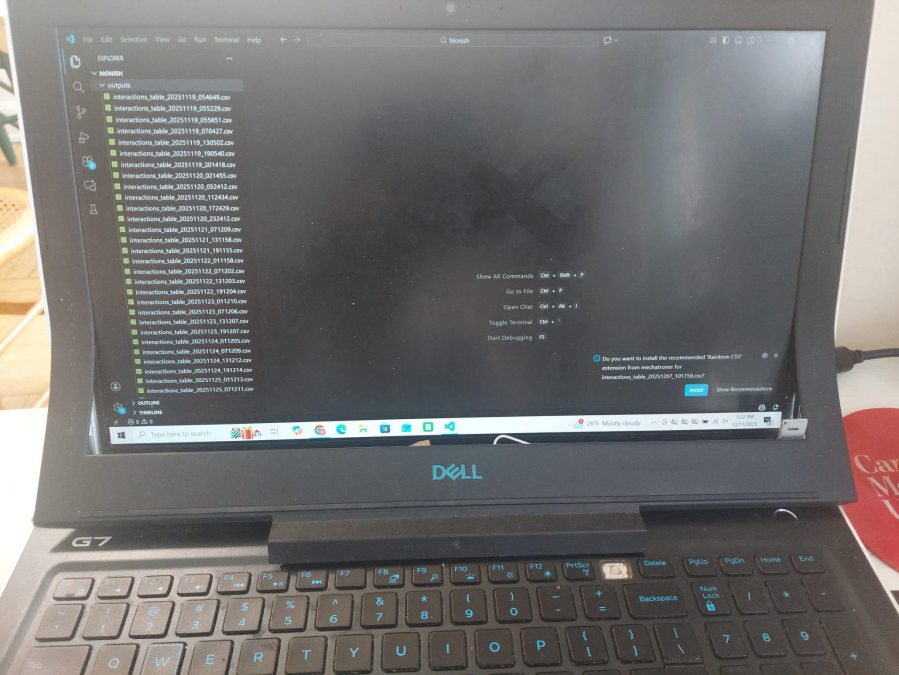
A separate VM aggregates logs, transfers updated interaction data every 6 hours, triggers retraining via Kubernetes CronJobs.

Kubernetes orchestrates two parallel deployments: Blue (80% traffic) for stability and Green (20% traffic) for gradual rollout of newly trained models, ensuring >90% uptime with seamless model updates.

We have used a private docker repository for application versioning, where as data is stored on the separate VM and the models and training logs are kept on the main VM.







Challenges Overcome

- Resolved multiple integration issues across microservices and pipelines, including container-to-container communication and Kubernetes service discovery.
- Fixed blocked traffic caused by misconfigured VM networking by setting up proper port forwarding from the VM's public IP to the Kubernetes internal load balancer port restoring stable access.
- Encountered a severe issue where modified IPTables rules broke outbound DNS resolution



Reflection and Lessons Learnt

- Building the model was the easy part ensuring reliability, observability, and real-time monitoring in production required far more effort and became the true challenge.
- Keeping detailed history of models, data, and deployment versions significantly reduced debugging time and made the entire system more resilient and transparent.





04 | Milestone

Fairness

We evaluated fairness using engagement-based disparity ratios instead of accuracy-based metrics.

This is unusual because most fairness evaluations focus on predictive accuracy, but we measured real user experience fairness—interactions per user and content diversity—to ensure the system treats demographic groups equitably at the product level, not just the model level.



Feedback Loop

	Popularity Reinforcement	Cold-start User Narrowing
Definition	A loop where popular movies receive higher ranking due to rich interaction histories -> more exposure, watches, reinforcement	A loop where the model anchors too heavily on a new users' earliest interactions - often driven by exposure rather than preference
Negative impact	Shrinks catalog diversity, suppresses long-tail content, stagnates personalization, unfairly underserves users with niche tastes	Misclassifies early clicks as preferences, traps users in narrow content clusters, reinforces exposure bias
Mitigation	Break popularity cycle by adding diversity constraints, injecting exploration items, and down-weighting popularity so the model does not overfit to high-frequency titles	Prevent early overfitting by using a cold-start exploration mode, down-weighting first interactions, and assigning new users to broad warm-start clusters.

The background features a dark blue field with intricate, flowing patterns of white and light blue dots, resembling particle trails or data streams. A bright, horizontal orange and yellow glow is positioned in the upper-middle section, creating a focal point. The overall aesthetic is futuristic and dynamic.

Team Reflection

Reflection on the Recommendation Service

Challenge

Coordinating the full production-like pipeline (data ingestion → retraining → deployment) was difficult, especially managing Kubernetes, PVCs, and containerized services.

Unstable Aspects Needing More Investment

Online metrics like CTR and diversity remain low, and cold-start quality is unstable. Monitoring, schema validation, and drift detection exposed underlying data issues. Security risks and scalability still require significant hardening

What We Would Improve with More Time

Strengthen authentication + rate limiting, replace pickle with secure formats, and improve monitoring with automated alerts. Enhance exploration and recency-based feature engineering to boost accuracy and user engagement.

If Starting Over

Integrate security, fairness, provenance tracking, and testability from day one. Prioritize online metrics earlier and design stronger CI/CD boundaries. Use canary/blue-green releases to reduce risk of bad updates.

Reflection on Teamwork

What Went Well

Milestone 1 ran smoothly; divide & conquer worked because tasks were independent. Clear responsibility areas prevented duplicated work. Communication, PR reviews, and CI all helped catch issues early.

What Went Less Well

In Milestone 2, starting late created integration and reliability issues. Distributed tools (VMs, PVCs, Kubernetes, monitoring) made testability harder than expected. Tight dependencies required heavy coordination, and scheduling conflicts slowed progress.

Challenges Across Milestones

Troubleshooting distributed systems required joint debugging rather than independent work. Monitoring, data validation, and drift detection revealed issues we didn't anticipate. Milestone 3 especially required deeper collaboration due to interdependent pipelines.

What We'd Change / Lessons Learned

Start earlier on complex milestones and explicitly track technical debt. Map dependencies upfront, add mid-week syncs, and assign clearer ownership. Monitoring is harder than modeling—production engineering requires more iteration than expected.

Thank You!
Any Questions?

